

1. UVOD I PROJEKTNII ZADATAK

U modernim IoT sistemima, izbor komunikacionog protokola na relaciji edge-to-cloud direktno utiče na performanse, latenciju i potrošnju hardverskih resursa. Ovaj projekat se bavi razvojem i komparativnom analizom tri dominantna modela sinhronne komunikacije:

REST (Representational State Transfer) – tradicionalni arhitektonski stil koji koristi HTTP/1.1 i tekstualni JSON format.

gRPC (Google Remote Procedure Call) – savremeni framework visokih performansi zasnovan na HTTP/2 protokolu i binarnoj Protobuf serijalizaciji.

GraphQL – upitni jezik za API koji klijentima omogućava dinamičko definisanje strukture odgovora preko HTTP-a, eliminišući problem prevelikog dohvaćanja podataka (over-fetching).

Sistem simulira realno okruženje u kojem prenosivi senzori (wearable devices) neprekidno šalju biometrijske podatke sportista (puls, brzina, troosni akcelerometar i troosni žiroskop) ka cloud backendu.

2. ARHITEKTURA SISTEMA I IMPLEMENTACIJA

Celokupni sistem je u potpunosti kontejnerizovan pomoću Docker-a i sastoji se od jedne centralne baze podataka i tri nezavisna mikroservisa projektovana da izoluju ponašanje svakog protokola.

2.1. Podsistem baze podataka (PostgreSQL)

Za skladištenje vremenskih serija senzorskih podataka korišćena je PostgreSQL baza podataka. Kako bi se obezbedila stabilnost pod visokim stepenom konkurentnosti, implementirane su napredne tehnike indeksiranja:

Kompozitni B-Tree indeks nad kolonama (`athlete_id`, `recorded_at` DESC) za trenutno preuzimanje hronološke istorije pojedinačnih sportista.

BRIN (Block Range Index) nad kolonom `recorded_at` koji omogućava izuzetno brze agregacione upite nad istorijskim opsezima uz minimalno zauzeće diska.

2.2. REST Mikroservis (Python / FastAPI)

Razvijen u Python-u uz pomoć asinhronog frameworka FastAPI. Komunikacija se odvija standardno preko HTTP/1.1 protokola, a podaci se razmenjuju kao sirovi JSON stringovi. Kompletan API je u potpunosti dokumentovan OpenAPI (Swagger) specifikacijom na endpointu `/docs`.

- `POST /api/v1/metrics` – Upis novih senzorskih zapisa.
- `GET /api/v1/metrics/selective/{athlete_id}` – Povlačenje filtriranih kolona za klijente.
- `GET /api/v1/metrics/aggregation` – Generisanje teških analitičkih izveštaja.

2.3. gRPC Mikroservis (Node.js)

Implementiran u Node.js okruženju korišćenjem native binarne serijalizacije preko HTTP/2 sa ugrađenim multipleksiranjem konekcija. Struktura servisa i poruka definisana je u fajlu `athlete.proto`

2.4. GraphQL Mikroservis (Node.js)

Zasnovan na Apollo Server-u unutar Node.js-a. Arhitektura je podeljena na deklarativnu šemu tipova (`typeDefs.js`) i izvršnu logiku (`resolvers.js`).

3. REZULTATI EVALUACIJE PERFORMANSI (LOAD TEST)

Evaluacija performansi izvršena je simulacijom masovnog opterećenja do **500 istovremenih virtuelnih korisnika (VUs)** pomoću alata **k6**. Sva tri scenarija su uspešno prošla sve integrisane validacione provere sa **100% uspešnošću**.

3.1. Scenario A: High-Frequency Ingestion (Masovni upis)

Simulira proces u realnom vremenu gde stotine senzora istovremeno šalju pun paket od 10 metričkih vrednosti ka serveru.

- **Ukupan broj uspešnih operacija:** 18,302 iteracija
- **Prosečan broj zahteva u sekundi (RPS):** 285.54 req/s
- **Latencije za REST i GraphQL (HTTP):**
 - Prosečna (*avg*): 671.85 ms
 - Medijana (*med*): 524.00 ms
 - 95. percentil (*p95*): 1.78 s
- **Latencija za gRPC:**
 - Prosečna (*avg*): 223.14 ms
 - Medijana (*med*): 180.82 ms
 - 95. percentil (*p95*): 642.88 ms

3.2. Scenario B: Selective Monitoring (Selektivno čitanje)

Simulira klijentske aplikacije na terenu koje zahtevaju istoriju kretanja sportista, ali povlače optimizovan skup podataka (samo 2 kolone umesto svih 10).

- **Ukupan broj uspešnih operacija:** 10,129 iteracija
- **Prosečan broj zahteva u sekundi (RPS):** 157.16 req/s
- **Latencije za REST i GraphQL (HTTP):**
 - Prosečna (*avg*): 967.18 ms
 - Medijana (*med*): 781.32 ms
 - 95. percentil (*p95*): 2.88 s
- **Latencija za gRPC:**
 - Prosečna (*avg*): 103.89 ms
 - Medijana (*med*): 28.40 ms
 - 95. percentil (*p95*): 466.36 ms

3.3. Scenario C: Heavy Querying (Agregacija podataka)

Simulira generisanje složenih analitičkih izveštaja sa matematičkim funkcijama nad istorijskim podacima iz baze, što stavlja maksimalan pritisak na hardver.

- **Ukupan broj uspešnih operacija:** 3,186 iteracija
- **Prosečan broj zahteva u sekundi (RPS):** 49.03 req/s
- **Latencije za REST i GraphQL (HTTP):**
 - Prosečna (*avg*): 4.60 s
 - Medijana (*med*): 4.57 s
 - 95. percentil (*p95*): 9.29 s
- **Latencija za gRPC:**
 - Prosečna (*avg*): 3.47 s
 - Medijana (*med*): 3.32 s
 - 95. percentil (*p95*): 8.05 s

4. ANALIZA VELIČINE ODGOVORA (MREŽNI ASPEKT)

Merenjem pojedinačnih i grupnih zahteva u **Postman Console** za identičan set od 100 senzorskih zapisa, dobijene su egzaktne vrednosti veličine korisnog tereta (*payload*) na mreži:

- **REST (JSON): 34.2 KB** – Tekstualni JSON format zahteva ponavljanje naziva kolona u vidu ključeva za svaki pojedinačni objekat u nizu (npr. `"heart_rate": 85`), što stvara ogroman mrežni overhead.
- **GraphQL (JSON): 11.4 KB** – Zahvaljujući sposobnosti GraphQL-a da eliminiše nepotrebne kolone na zahtev klijenta (zadržane samo 2 od 10 kolona), veličina odgovora je redukovana za čak 66% u odnosu na REST, čineći ga odličnim za mobilne uređaje na lošoj vezi.
- **gRPC (Protobuf): 7.8 KB** – Najefikasniji rezultat na testu. Pošto ne koristi tekstualne ključeve već binarnu kompresiju zasnovanu na numeričkim tagovima polja iz `.proto` datoteke, gRPC šalje isključivo čiste sirove bajtove, trošeći drastično manje mrežnog saobraćaja.

5. MONITORING I ANALIZA POTROŠNJE PROCESORSKIH RESURSA

Tokom vršnog opterećenja od 500 korisnika, podaci prikupljeni preko **cAdvisor-a** i vizuelizovani u **Grafani** pokazuju kako proces serijalizacije i deserijalizacije podataka utiče na CPU i RAM kontejnera na procesorskom nivou:

5.1. Analiza CPU opterećenja

- **GraphQL Kontejner:** Zabeležio najveću prosečnu potrošnju CPU-a od **84%**. Uzrok leži u visokoj ceni parsiranja tekstualnog stringa upita, kreiranju apstraktnog sintaksnog stabla (AST) i proveru strukture u odnosu na definisanu šemu prilikom svake dolazne operacije.
- **REST (FastAPI) Kontejner:** Pokazao umerenu i stabilnu potrošnju CPU-a od oko **45%**. FastAPI efikasno hendluje asinhroni rad, ali se procesorski resursi troše na konverziju sirovih redova iz baze u tekstualni JSON string pre slanja.
- **gRPC Kontejner:** Pokazao izuzetnu hardversku efikasnost sa svega **18%** zauzeća CPU-a. Binarni Protobuf format mapira bajtove sa mreže direktno u radnu memoriju Node.js servisa, zaobilazeći skupi tekstualni parsing, što ga čini najjeftinijom operacijom na procesorskom nivou.

5.2. Analiza RAM otiska

- **GraphQL (Apollo Server):** ~112 MB (zbog keširanja šeme i AST stabla).
- **REST (FastAPI):** ~68 MB (lagan i optimizovan Python proces).
- **gRPC (Node.js):** ~44 MB (minimalan memorijski otisak usled binarne obrade podataka).

6. ZAKLJUČAK

Na osnovu empirijskih podataka, izvedeni su ključni zaključci za projektovanje IoT sistema:

1. **Za Edge-to-Cloud Ingestion: gRPC** je apsolutni pobednik sa najnižom latencijom (642.88ms u p95 naspram 1.78s kod REST-a), najmanjom potrošnjom CPU-a (18%) i najkompaktnijim paketom (7.8 KB), što direktno čuva bateriju i procesor IoT uređaja.
2. **Za aplikacije krajnjih korisnika: GraphQL** nudi vrhunsku mrežnu uštedu na klijentskoj strani (pad na 11.4 KB) eliminišući *over-fetching*, iako zahteva jače serverske resurse.
3. **Za analitiku velikih podataka:** U scenarijima sa teškim računskim upitima (Scenario C), performanse zavise primarno od optimizacije baze podataka (korišćenje BRIN indeksa), gde **REST** pruža stabilan i standardizovan interfejs za isporuku analitike.